南京工业大学

计算机与信息工程学院（人工智能学院）

上 机 报 告

（2026 - 06 学年第 学期）

课程名称：编译原理 授 课教 师 : 组员

组员 A

班 级： Compiler Principles

学 号： 202324405009

姓 名： Ameer subhan hussain

年 月

一、目的和要求（无需填写）

本上机实习对应课程目标 5，要求能够根据语言的文法定义，设计并实现一个编译程序（包括词法分析、语法分析和语义分析功能和一定的报错功能），并能够对设计思路进清晰的描述。

二、文法（无需填写）

<program> → <variable_declaration_part> <statement_part>
<variable_declaration_part> → <variable_declaration_part>
<variable_declaration_statement> ; | <variable_declaration_statement> ;
<variable_declaration_statement> → <variable_declaration> <identifier_list>
<variable_declaration> → int | string | bool
<identifier_list> → <identifier_list> , <identifier> | <identifier>
<identifier> → $ <letter> <letter_sequence> $
<statement_part> → <statement_part> ; <statement> | <statement>
<statement> → <assignment_statement> | <conditional_statement> | <loop_statement>
<assignment_statement> → <identifier> := <expression>
<expression> → <disjunction>
<disjunction> → < conjunction > | <disjunction> or < conjunction >
<conjunction > → < inversion > | < conjunction > and < inversion >
<inversion> → not <inversion> | <relational_expression>
<relational_expression> → <arithmetic_expression> <relational_operator>
<arithmetic_expression> | <arithmetic_expression>
<arithmetic_expression> → <term> | <arithmetic_expression> <addition_operator> <term>
<term> → <factor> | <term> <multiplication_operator> <factor>
<factor> → <identifier> | true | false | <string> | <integer_sequence> | (<expression>)
<relational_operator> → < | > | <> | >= | <= | ==
<addition_operator> → + | -
<multiplication_operator> → * | / | %
<string> → " <letter_sequence> "
<letter_sequence> → <letter_sequence> <digit> | <letter_sequence> <letter> | ε
<integer_sequence> → <integer_sequence> <digit> | <digit>
<conditional_statement> → if (<expression>) <nested_statement> else
<nested_statement>
<loop_statement> → while (<expression>) : <nested_statement>
<nested_statement> → <statement> ; | <compound_statement>
<compound_statement> → { <statement_part> }
<letter> → [a-z]
<digit> → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

三、词法分析程序的实现思路

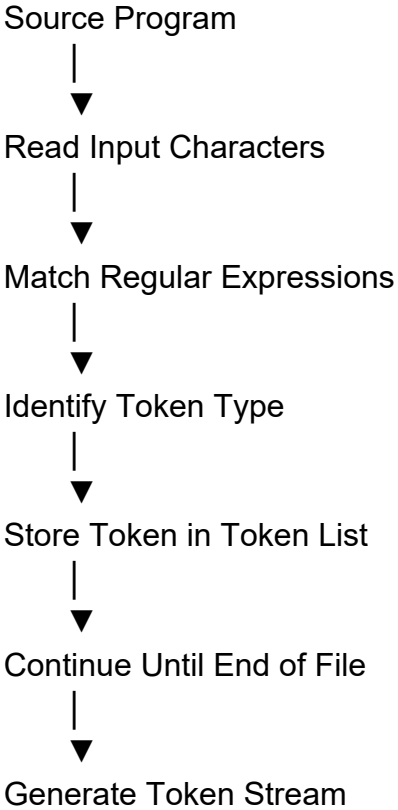
分析文法，写出所有的 后，上机编程，根据状 并其 （ 机的 有机。 单词对编码对应动 ，并画出 的状 态转换图实现 穷 动 终态） 对应 态转换图。然

1. Design Idea / Workflow

The lexical analyzer is the first phase of the compiler. Its task is to scan the source program character by character and convert the input stream into a sequence of tokens. Each token represents a meaningful language element such as keywords, identifiers, constants, operators, and delimiters.

The implementation is based on Finite Automata and Regular Expressions.

Workflow



2. Token Classification

Thelexical analyzer recognizes the following token categories:

Token Type	Example
Keywords	int, string, bool, if, else, while
Identifier	a, $name$
Number	10, 100, 999
Operators	+, -, *, /, :=
Relational Operators	>, <, >=, <=, ==, <>
Delimiters	; , ( ) { }
Boolean Constants	true, false

3. State Transition Idea

Identifier DFA

Identifiers follow the grammar:

$ letter letter* $

State transition:

Start
 |
 ▼
'$'
 |
 ▼
Letter
 |
Letter/Digit
 |
 ▼
'$'
 |
 ▼
Accept Identifier

Integer DFA
Start
 |
Digit
 |
Digit
 |
 ▼
Accept Integer

Assignment Operator DFA
Start
 |
':'
 |
'='
 |
 ▼
Accept :=

---

4. Program Implementation

The lexical analyzer was implemented in Python using the re (Regular Expression) library. Each token type is represented by a regular expression pattern. The analyzer scans the input string from left to right and matches the longest valid token.

```
import re

TOKENS = [
    ('NUMBER',                  r'\d+'),
    ('IDENTIFIER',   r'\$[a-zA-Z]+\$'),
    ('ASSIGN', r':='),
    ('PLUS', r'\+'),
    ('MINUS', r'-'),
    ('MULT', r'\*'),
    ('DIV', r'/'),
    ('LPAREN', r'\('),
    ('RPAREN',                    r'\)'),
    ('SEMICOLON', r';'),
    ('IF', r'if'),
```

```
    ('ELSE', r'else'),
    ('WHILE', r'while'),
    ('INT', r'int'),
    ('STRING', r'string'),
    ('BOOL', r'bool'),
    ('TRUE', r'true'),
    ('FALSE', r'false'),
    ('SKIP', r'[ \t\n]+'),
    ('COMMA', r','),
    ('LBRACE', r'\{'),
    ('RBRACE', r'\}'),
    ('GT', r'>'),
    ('LT', r'<'),
    ('EQ', r'=='),
    ('NEQ', r'<>'),
    ('GE', r'>='),
    ('LE', r'<=')
]

def tokenize(code):
    tokens = []
    while code:
        for token_type, pattern in TOKENS:
            match = re.match(pattern, code)
            if match:
                text = match.group(0)
                if token_type != 'SKIP':
                    tokens.append((token_type, text))
                code = code[len(text):]
                break
        else:
            raise Exception(
                f"Lexical Error: Invalid token near '{code[:10]}'"
            )
    return tokens
```

5. Experimental Result / Proof of Execution

TestInput

```
int $a$;
$a$ := 10;
```

Program Output

```
('INT', 'int')
('IDENTIFIER', '$a$')
('SEMICOLON', ';')

('IDENTIFIER', '$a$')
('ASSIGN', ':=')
('NUMBER', '10')
('SEMICOLON', ';')
```

Another Test Case

Input:

```
bool $flag$;
$flag$ := true;
```

Output:

```
('BOOL', 'bool')
('IDENTIFIER', '$flag$')
('SEMICOLON', ';')

('IDENTIFIER', '$flag$')
('ASSIGN', ':=')
('TRUE', 'true')
('SEMICOLON', ';')
```

Error Detection Test

Input:

```
int @abc;
```

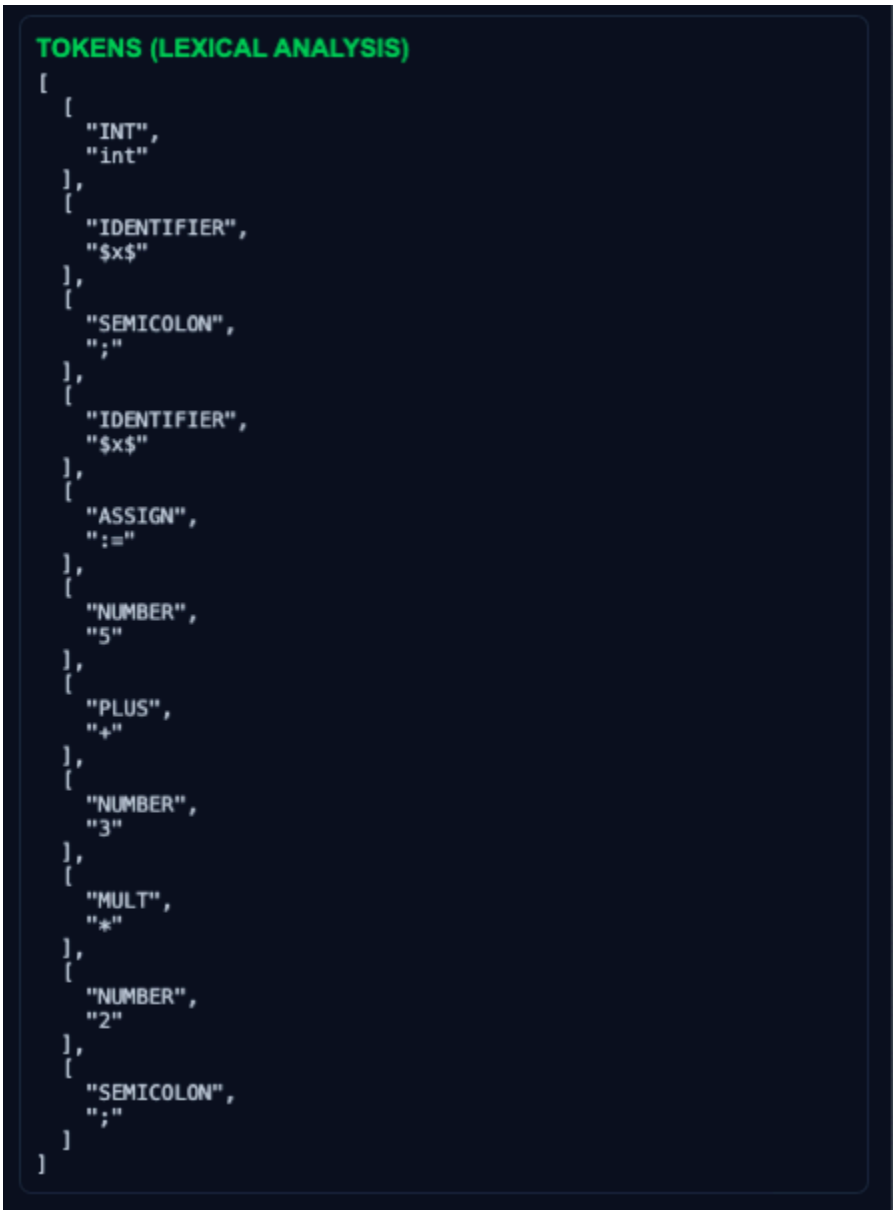
Output:

```
Lexical Error:
Invalid token near '@abc'
```

Conclusion

The lexical analyzer successfully recognizes keywords, identifiers, constants, operators,

delimiters, and boolean values. Invalid symbols are detected and reported as lexical errors. The generated token stream serves as input for the syntax analysis phase.



四、语法分析程 实现 思路

序的

1. 在第二题文法中划去多余非终结符和终结符（直接在第二题上改）。然后，从剩余产生式 中 挑出存在左递归或回溯的产生式，在下方写出消除左递归和回溯的过程和结果。

Example 1: Statement Part

Original Grammar

<statement_part>
→ <statement_part> ; <statement>
| <statement>

This production contains direct left recursion.

After Removing Left Recursion

<statement_part>
→ <statement> <statement_part'>

<statement_part'>
→ ; <statement> <statement_part'>
| ε

Example 2: Identifier List

Original Grammar

<identifier_list>
→ <identifier_list> , <identifier>
| <identifier>

Rewritten Grammar

<identifier_list>
→ <identifier> <identifier_list'>

<identifier_list'>
→ , <identifier> <identifier_list'>
| ε

Example 3: Arithmetic Expression

Original Grammar
<arithmetic_expression>
→ <arithmetic_expression> + <term>
| <arithmetic_expression> - <term>
| <term>

Rewritten Grammar
<arithmetic_expression>
→ <term> <expression'>

<expression'>
→ + <term> <expression'>
| - <term> <expression'>
| ε

Example 4: Term

Original Grammar
<term>
→ <term> * <factor>
| <term> / <factor>
| <factor>

Rewritten Grammar
<term>
→ <factor> <term'>

<term'>
→ * <factor> <term'>
| / <factor> <term'>
| ε

Final Grammar Used in the Parser

The parser implementation was based on the following LL(1) style grammar:

program
→ declarations statements

declarations
→ declaration ; declarations
| ε

declaration
→ type identifier_list

identifier_list
→ identifier
{ , identifier }

statement
→ assignment
| if_statement
| while_statement
| block

assignment
→ identifier := expression

if_statement
→ if (expression) statement else statement

while_statement
→ while (expression) statement

block
→ { statements }

expression
→ term { (+|-) term }

term
→ factor { (*|/) factor }

factor
→NUMBER
|IDENTIFIER
|TRUE
|FALSE
|(expression)

2. 针对改写过的文法，写出需要的产生式的 集。然后上机编程，实现每条产生 式的语法 分析子程序。

2. FIRST Sets

The FIRST sets were calculated for the rewritten grammar.

FIRST(program)

{ INT, STRING, BOOL }

FIRST(declaration)

{ INT, STRING, BOOL }

FIRST(identifier_list)

{ IDENTIFIER }

FIRST(statement)

{

IDENTIFIER,

IF,

WHILE,

LBRACE

}

FIRST(assignment)

{ IDENTIFIER }

FIRST(if_statement)

{ IF }

FIRST(while_statement)

{ WHILE }

FIRST(block)

{ LBRACE }

FIRST(expression)

{

NUMBER,

IDENTIFIER,

LPAREN,

TRUE,

FALSE

}

FIRST(term)

{

NUMBER,

IDENTIFIER,

LPAREN,

TRUE,

FALSE

}

FIRST(factor)

{

NUMBER,

IDENTIFIER,

LPAREN,

TRUE,

FALSE

}

Parser Design Workflow

The parser receives tokens generated by the lexical analyzer and checks whether the token sequence follows the grammar rules.

Source Program



Lexical Analyzer



Token Stream



Recursive Descent Parser

AST Generation

|

▼

Syntax Checked Program

Parser Implementation

The parser was implemented using Recursive Descent Parsing. Each non-terminal symbol of the grammar was implemented as a separate Python function.

Grammar Rule	Parser Function
program	parse_program()
declarations	parse_declarations()
declaration	parse_declaration()
identifier_list	parse_identifier_list()
statement	parse_statement()
assignment	parse_assignment()
if_statement	parse_if()
while_state	parse_while()

ment	
block	parse_block()
expression	parse_expressio n()
term	parse_term()
factor	parse_factor()

Experimental Result

Input Program

```
int $a$;

$a$ := 10;

while ($a$)
{
  $a$ := $a$ - 1;
}
```

Parsing Process

```
parse_program()
│
├── parse_declarations()
│
└── parse_statements()
```

```
|
├── parse_assignment()
|
└── parse_while()
    |
    └── parse_block()
```

Result

Parsing Successful

Abstract Syntax Tree Generated

No Syntax Errors Detected

Conclusion

The original grammar was transformed into an LL(1)-compatible grammar by removing left recursion. FIRST sets were computed for predictive parsing. A Recursive Descent Parser was then implemented in Python, where each production rule was represented by a dedicated parsing function. The parser successfully recognizes declarations, assignments, conditional statements, loop statements, blocks, and arithmetic expressions while detecting syntax errors when grammar rules are violated.

五、语义 分析程序的 实现 思路

1. 写出翻译成四元式时，需要考虑的声明语句、赋值语句、表达式、条件语句和 环语 句的 参 循 数传递方向（形式不限，如伪代码、属性文法或语法树）。

V. Implementation Approach of the Semantic Analysis Program

Semantic analysis is performed after syntax analysis. The semantic analyzer checks whether variables are declared before use and maintains a symbol table. The intermediate code generator translates expressions into quadruples, while the parser constructs an Abstract Syntax Tree (AST).

---

1. Translation into Quadruples

A quadruple has the following format:

(operator, argument1, argument2, result)

Example:

a := b + c

Generated quadruples:

(+, b, c, t1)
(:=, t1, -, a)

(a) Declaration Statement

Source Statement
int a, b

Parameter Passing
Declaration

Variable Type



Identifier List



Insert into Symbol Table

Pseudocode

```
for each identifier:
  insert(identifier, type)
```

Symbol Table Result

Identifier	Type
a	int
b	int

(b) Assignment Statement

Source Statement

```
$a$ := 10
```

Parameter Passing

```
Assignment
├── Identifier
└── Expression
```

Semantic Action

```
check(identifier)

value = expression.result

generate(:=, value, -, identifier)
```

Quadruple

```
(:=, 10, -, $a$)
```

(c) Expression

Source Statement
a + b

Parameter Passing
Expression
├── Left Operand
├── Operator
└── Right Operand

Pseudocode
left = generate(left)
right = generate(right)

temp = new_temp()

emit(op, left, right, temp)

return temp

Quadruple
(+, a, b, t1)

This corresponds directly to the `BinaryOp` handling in `intermediate.py`.

(d) Conditional Statement

Source Statement
```
if  ($a$)
{
  $b$ := 1;
}
else
{
  $b$ := 0;
}
```

Parameter Passing
If Statement

```
├── Condition
├── Then Branch
└── Else Branch
```

Semantic Flow

```
evaluate(condition)

if true
   execute then_branch
else
   execute else_branch
```

Typical Quadruples

```
(IF, $a$, -, L1)
(GOTO, -, -, L2)

L1:
(:=, 1, -, $b$)

L2:
(:=, 0, -, $b$)
```

(e) Loop Statement

Source Statement

```
while ($a$)
{
   $a$ := $a$ - 1;
}
```

Parameter Passing

```
While Statement
├── Condition
└── Body
```

Semantic Flow

```
repeat:
   evaluate condition

   if false
      exit
```

execute body

goto repeat

Typical Quadruples
L1:
(IF, a, -, L2)
(GOTO, -, -, L3)

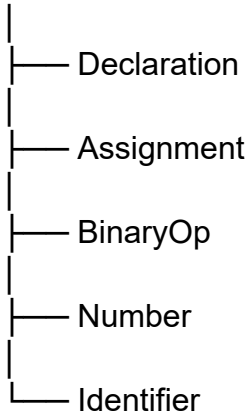
L2:
(-, a, 1, t1)
(:=, t1, -, a)
(GOTO, -, -, L1)

2. 写出翻译成抽象语法树时，需要考虑的声明语句、赋值语句、表达式、条件语句和循 的参数传递方向（形式不限，如伪代码、属性文法或语法树）。

2. Translation into Abstract Syntax Tree (AST)

The parser constructs an Abstract Syntax Tree during syntax analysis. Each grammar production is translated into a node object.

AST Node Hierarchy

Program


This hierarchy corresponds to the classes implemented in `ast_nodes.py`.

(a) Declaration Statement AST

Source Statement
int a

AST
Declaration
│
├── Type: int
└── Identifier: a

Created Node
Declaration("int", ["a"])

(b) Assignment Statement AST

Source Statement
a := 10

AST
```
    :=
   /  \
 $a$10
```

Created Node
Assignment("a", Number(10))

(c) Expression AST

Source Statement
a + b

AST
```
     +
```

```
    / \
  $a$$b$
```

Created Node

```
BinaryOp(
  Identifier("$a$"),
  "+",
  Identifier("$b$")
)
```

(d) Complex Expression AST

Source Statement

```
$a$ + $b$ * $c$
```

AST

```
      +
    /   \
  $a$    *
       / \
     $b$$c$
```

Explanation

Multiplication has higher precedence, therefore the subtree for * is created before the + node.

(e) Program AST

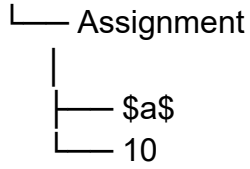
Source Program

```
int $a$;

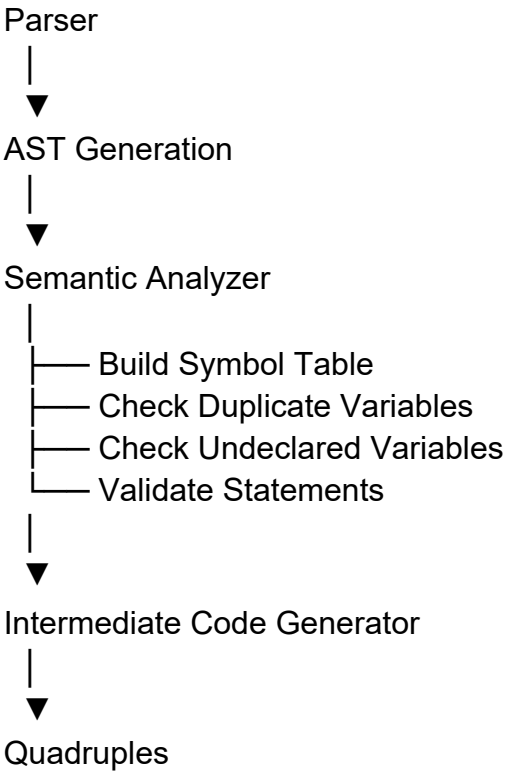
$a$ := 10;
```

AST

```
Program
│
├── Declaration(int, [$a$])
│
```



Semantic Checking Workflow



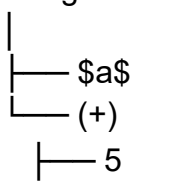
Experimental Result

Input

```
int $a$;

$a$ := 5 + 3;
```

AST



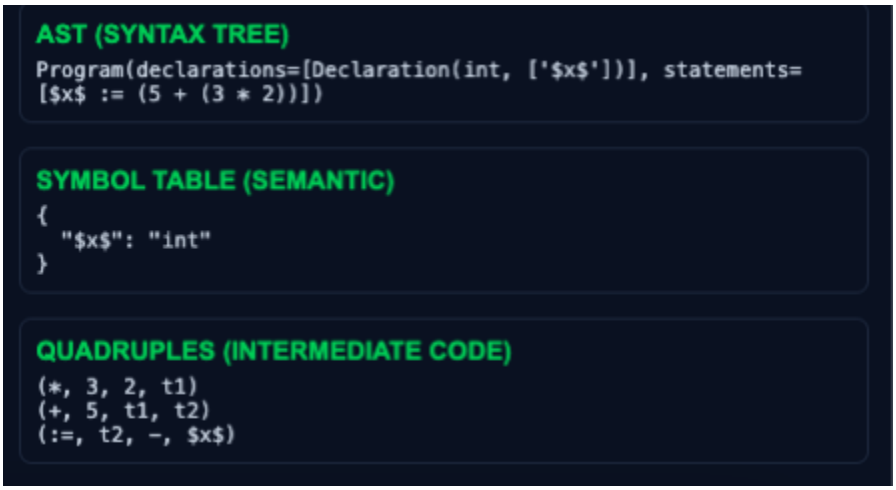
└── 3

Generated Quadruples
(+, 5, 3, t1)
(:=, t1, -, a)

Semantic Result
Symbol Table:

a → int

No Semantic Errors



六、从下述 问题 中勾选至少一题，进行论述

- □ 实现鲋错　功能及设计思路
- □ 实现的拓广功能及设计思路
- □ 检验翻译结果是否正确的思路与方法
- □ Java或C语言编译器的分析与理解
- □ 编译器性能斌实验的设计评与　估
- ☑ 上机过程中遇到的最大的难题、解决步骤和方

VI. The Biggest Difficulty Encountered During the Hands-on Practice, the Solution Steps and Methods

Introduction

During the implementation of the compiler project, the most challenging part was integrating the lexical analyzer, parser, semantic analyzer, abstract syntax tree generation, and intermediate code generation into a complete compilation process. Although each module could be implemented independently, making them work together correctly required careful debugging and testing.

Main Difficulty 1: Designing a Recursive Descent Parser

The original grammar contained productions that were not directly suitable for Recursive Descent Parsing. Some productions exhibited left recursion, which can cause infinite recursion during parsing.

For example:

Expression → Expression + Term
　　| Term

If implemented directly, the parser repeatedly calls itself and never terminates.

Solution

The grammar was rewritten by eliminating left recursion.

Expression → Term Expression'

Expression' → + Term Expression'
| - Term Expression'
| ε

After rewriting the grammar, separate parser functions such as`parse_expression()`, parse_term(), and parse_factor() were implemented successfully

---

Main Difficulty 2: Managing Operator Precedence

Expressions such as:

a + b * c

must be interpreted as:

a + (b * c)

instead of:

(a + b) * c

Solution

The parser was divided into multiple levels:

parse_expression()
parse_term()
parse_factor()

This structure naturally enforces operator precedence because multiplication and division are processed before addition and subtraction.

Main Difficulty 3: Semantic Checking

Another challenge was ensuring that variables are declared before they are used.

For example:

a := 10;

If a has not been declared previously, the compiler should report an error.

Solution

A symbol table was implemented in the semantic analyzer.

Symbol Table

Identifier	Type
a	int
b	string

Whenever a variable is used, the semantic analyzer checks whether it exists in the symbol table. If it is missing, a semantic error is generated.

Example:

Semantic Error:
'a' not declared

Main Difficulty 4: Generating Intermediate Code

Generating quadruples for complex expressions required recursive processing of expression trees.

For example:

a := b + c * d

Solution

Temporary variables were introduced.

Generated quadruples:

(*, c, d, t1)
(+, b, t1, t2)
(:=, t2, -, a)

The use of temporary variables simplified intermediate code generation and preserved operator precedence.

Verification Method

The correctness of the compiler was verified using several test programs.

Test Case 1

Input:

int a;

a := 10;

Result:

Lexical Analysis: Passed
Syntax Analysis: Passed
Semantic Analysis: Passed

Test Case 2

Input:

a := 10;

Result:

Semantic Error:
'a' not declared

Test Case 3

Input:

int a;

a := 5 + 3 * 2;

Generated Quadruples:

(*, 3, 2, t1)
(+, 5, t1, t2)
(:=, t2, -, a)

The output confirmed that operator precedence was handled correctly.

Conclusion

The most difficult aspect of this project was coordinating all compiler phases and ensuring that they worked together correctly. The problems of left recursion, operator precedence, semantic checking, and intermediate code generation were solved through grammar rewriting, recursive descent parsing, symbol table management, and the use of temporary variables. Through repeated testing and debugging, a working compiler front-end capable of lexical analysis, syntax analysis, semantic analysis, AST generation, and quadruple generation was successfully developed.

七、自我评价与小结

该部分不算入成绩，如实填写即可（选择题均为单选；若选择不同意，可以简单标注一下 原因，作为教研参考）

组员 A：

1. 我基本掌握了根据语言的文法定义，设计并实现一个编译程序的方法（比如考试时给我 一个其他的文法，我应该可以设计出一个简单的实现方法）

☑同意 □基本同意 □不确定 □有点不同意 □不同意

2. 我基本能够对设计思路进描述（比如有人询问我的思路，我应该可以给他解释明白）

☑同意 □基本同意 □不确定 □有点不同意 □不同意

3. 在上机之前，我对理论课的知识的掌握应该算是一知半解，也许能够解题，但是不明白 具体含义。

□同意 ☑基本同意 □不确定 □有点不同意 □不同意

4. 通过上机实践，一定程度上加深了我对理论知识的理解。

☑同意 □基本同意 □不确定 □有点不同意 □不同意

5. 上机实践中我可能遇到了挫折，虽然我花费了额外的时间解决这些问题，但是当我成功 运行程序后，我获得了成就感。

☑同意 □基本同意 □不确定 □有点不同意 □不同意

6. 通过上机实践，我发现自己大学期间学会了一些知识和技能，也发现了自己不会的东西 更多，我意识到学无止尽，同时意识到自己潜力无穷。

☑同意 □基本同意 □不确定 □有点不同意 □不同意

7. 写出其他的心得与体验：

组员 B：

7. 我基本掌握了根据语言的文法定义，设计并实现一个编译程序的方法（比如考试时给我 一个其他的文法，我应该可以设计出一个简单的实现方法）

□同意 □基本同意 □不确定 □有点不同意 □不同意

8. 我基本能够对设计思路进描述（比如有人询问我的思路，我应该可以给他解释明白）

□同意 □基本同意 □不确定 □有点不同意 □不同意

9. 在上机之前，我对理论课的知识的掌握应该算是一知半解，也许能够解题，但是不明白 具体含义。

□同意 □基本同意 □不确定 □有点不同意 □不同意

10. 通过上机实践，一定程度上加深了我对理论知识的理解。

□同意 □基本同意 □不确定 □有点不同意 □不同意

11. 上机实践中我可能遇到了挫折，虽然我花费了额外的时间解决这些问题，但是当我成功 运行程序后，我获得了成就感。

□同意 □基本同意 □不确定 □有点不同意 □不同意

12. 通过上机实践，我发现自己大学期间学会了一些知识和技能，也发现了自己不会的东西更多，我意识到学无止尽，同时意识到自己潜力无穷。

□同意 □基本同意 □不确定 □有点不同意 □不同意

7. 写出其他的心得与体验：